Deductive Verification of Legal Contracts

Reiner Hähnle[1[0000−0001−8000−7613]] and Cosimo Laneve[2[0000−0002−0052−4061]]

[1] Department of Computer Science, TU Darmstadt, Germany
reiner.haehnle@tu-darmstadt.de
[2] DISI, University of Bologna, Italy
cosimo.laneve@unibo.it

Abstract. We enable deductive verification for *Stipula*, a domain-specific language for legal contracts, by systematically translating contracts into `Java` programs annotated with `Java Modeling Language` specifications, and subsequently proving them using a deductive verification tool. A central challenge of the translation lies in representing event-driven and time-dependent behaviour within a static specification framework. To address this, we introduce a dispatch table that records schedulable events together with their triggering times. The technique is presented for acyclic *Stipula* contracts. We also extend the proposed technique to cyclic contracts, outlining the additional challenges and the conditions under which the approach remains applicable.

Keywords: Formal Verification · Stipula Language · Translation-based Verification · Deductive Verification.

1 Introduction

Stipula is a domain-specific programming language designed to model legal contracts with formally enforceable properties, particularly those governing asset transfers, obligations, and time-dependent rights [8]. Contracts written in *Stipula* combine declarative clauses with executable semantics, yielding models whose behaviour evolves dynamically over time. To facilitate the contract lifecycle, a supporting toolchain has been developed that has been integrated into a unified workbench, enabling legal practitioners and developers in writing, testing, debugging, and managing legal contracts [13, 21]. While this infrastructure provides practical support for contract development and experimentation, it currently lacks advanced deductive verification capabilities.

This paper complements the existing *Stipula* workbench with a formal verification layer capable of establishing both partial and total correctness properties of legal contracts. Our approach relies on a systematic translation of *Stipula* contracts into `Java` programs annotated with `Java Modeling Language` (`JML`) specifications, thereby reducing the verification of legal contracts to the discharge of proof obligations within a mature and well-established formal methods framework. Concretely, we use KeY [4] as a general-purpose deductive verification system.

A central challenge in this setting is achieving sufficient precision in the representation of execution behaviour. Legal contracts typically exhibit *dynamic* features – notably the scheduling of events at future times – while `JML` specifications are inherently *static*. Modelling time and deferred events within a deductive verification framework, therefore, requires an explicit and carefully structured representation of the contract's temporal evolution. Unlike our preliminary study [11], where time and events were treated in an abstract manner, the present work fully incorporates timed events into the translation. This is challenging because the underlying semantic models of *Stipula* contracts are infinite-state due to the unbounded progression of time and the generation of future events.

To represent the behaviour of *Stipula* using static method contracts, we exploit the finite-state structure defined by the contract's underlying automaton, in line with *Stipula*'s state-based programming style. Starting from this automaton, we systematically construct all sequences of clauses that are legally admissible according to the contract specification. This construction yields an explicit representation of the contract's behavioural space, abstracting away from concrete executions while preserving the permitted control-flow structures. A key component of this process is the dispatch table, which records the events that can be scheduled together with their trigger times. This representation plays a central role in verification: the soundness and completeness of the properties that can be established depend directly on how precisely the admissible execution paths are captured.

We first present the technique for acyclic *Stipula* contracts. In this setting, we formally define the translation, describe the construction of the dispatch table and the generation of execution paths, and show how partial-correctness properties of contracts can be reduced to proof obligations discharged automatically by KeY. For acyclic contracts, the behavioural space is finite and can be represented with high precision.

We subsequently extend our approach to cover cyclic contracts. The translation devised for the acyclic setting can be generalised to contracts featuring *separate cycles*: cycles are not nested, even if multiple paths between their entry/exit states are permitted. Here, each cycle is mapped to a bounded `Java` loop iteration, whose termination is enforced through an explicit counter and a predetermined upper bound. This encoding makes the verification task possible for a deductive verification tool, while deliberately introducing a controlled over-approximation of the contract's potentially unbounded behaviour. As customary in deductive verification, the presence of loop iterations requires suitable invariants. Accordingly, while the structural aspects of the translation are fully automated, the user may be required to provide contract-specific invariants to enable KeY to complete the proof obligations generated by cyclic constructs.

Overall, the main contributions of this work are as follows:

- *A principled encoding of time-dependent contractual behaviour.* We devise a novel and adequate data structure, the *dispatch table*, for representing time constraints and event scheduling within a deductive verification setting.

```
stipula C {
    assets h̄
    fields x̄
    agreement(Ā) {
        Ā_1 : x̄_1
        ...
        Ā_n : x̄_n
    } ⇒ @Q
    F
}
```

Functions	F ::=	$_ \mid \texttt{@Q}\,\texttt{A} : \texttt{f}(\overline{\texttt{y}})[\overline{\texttt{k}}]\,(E)\{\,S\,;\,W\,\} \Rrightarrow \texttt{@Q}'\,F$
Prefixes	P ::=	$E \rightarrow \texttt{x} \mid E \rightarrow \texttt{A} \mid E \multimap \texttt{h}, \texttt{h}'$
		$\mid E \multimap \texttt{h}, \texttt{A}$
Statements	S ::=	$_ \mid P\,S \mid \textbf{if}\,(E)\,\{\,S\,\}\,\textbf{else}\,\{\,S\,\}\,S$
Events	W ::=	$_ \mid \textbf{now} + \texttt{t} \gg \texttt{@Q}\,\{\,S\,\} \Rrightarrow \texttt{@Q}'\,W$
Expressions	E ::=	$v \mid \textbf{now} \mid X \mid E\,\texttt{op}\,E \mid \texttt{nop}\,E$

Fig. 1. Syntax of *Stipula*

- *A systematic and extensible translation framework.* We define a formal, fully specified translation from legal contracts written in *Stipula* into `Java` programs annotated with `JML` specifications. The translation is not limited to acyclic interaction patterns, but also covers separate (non-nested) cycles, thereby addressing a wide range of realistic contractual behaviour.
- *A sound verification methodology based on mainstream deductive technology.* We establish the soundness of the translation, ensuring that properties verified on the generated `JML`/`Java` code by a state-of-the-art deductive verification system such as KeY correctly reflect the semantics of the original contract.

The paper is organised as follows. Section 2 contains the necessary background about *Stipula* and KeY. Section 3 defines the translation function for *Stipula* contracts; the translation covers both acyclic contracts and those with cycles. Section 4 analyses the management of timed events with the dispatch table and the generation of execution paths; Section 5 extends the analysis to cyclic behaviour. Section 6 briefly reviews related efforts in formal verification of legal contracts and Section 7 concludes the paper and outlines directions for future work.

2 Background

To set the stage for our translation-based verification method, we briefly review the relevant background. Section 2.1 presents the *Stipula* language, emphasising its constructs for assets, states, and timed events. Section 2.2 introduces the KeY system, which we employ as the verification backend.

2.1 *Stipula*

A *Stipula* contract consists of a set of parties, states, assets, fields, and a set of functions and events. The declaration of a contract is defined in Figure 1, where `C` is the name of the contract, $\overline{\texttt{h}}$ and $\overline{\texttt{x}}$ are the *assets* and *fields*, respectively, $\overline{\texttt{A}}$ are the *parties*. The **`agreement`** construct declares the parties that set the initial value of the fields. For example, if the agreement has three parties $\texttt{A}_1, \texttt{A}_2, \texttt{A}_3$ and the contract has two fields $\texttt{x}_1, \texttt{x}_2$, if it declares $\texttt{A}_1 : \texttt{x}_1$ and $\texttt{A}_2, \texttt{A}_3 : \texttt{x}_2$ then $\texttt{x}_1$ will

be set by $\mathtt{A}_1$ and $\mathtt{x}_2$ will be set upon agreement on the value between $\mathtt{A}_2$ and $\mathtt{A}_3$. When the agreement is concluded, the initial state of the contract is set and the parties may invoke a function in F.

A *function* f declared as $\mathtt{@Q\,A} : \mathtt{f}(\overline{\mathtt{y}})[\overline{\mathtt{k}}]\,(E)\{\,S;\,W\,\} \Rightarrow \mathtt{@Q}'$ can be invoked by a party A if the contract is in state Q and the guard E is *true*. The names $\overline{\mathtt{y}}$ and $\overline{\mathtt{k}}$ are the formal parameters of f; they are kept separate because $\overline{\mathtt{y}}$ are *field* values while $\overline{\mathtt{k}}$ are *asset* quantities.

Function bodies are *statements* S followed by *events* W. The former include value transfers, asset movements, conditional logic, and field assignments. *Stipula* distinguishes between different transfer operations: field updates and messages use the symbol $\rightarrow$ with the usual semantics of assignment, while asset transfers, called *moves*, use the linear operator $\multimap$ to emphasise the conservation semantics. For instance, an expression like "$1 \multimap \mathtt{wallet}, \mathtt{Seller}$" denotes exclusive transfer of a unit in wallet to the Seller and, *at the same time*, the decrease by 1 of wallet. In contrast "$\mathtt{wallet} \rightarrow \mathtt{Seller}$" specifies that the value currently stored in wallet is communicated to Seller, while leaving the state of wallet unchanged. It is common practice in *Stipula* to use the abbreviations $\mathtt{h} \multimap \mathtt{h}'$ and $\mathtt{h} \multimap \mathtt{A}$, which stand for $\mathtt{h} \multimap \mathtt{h}, \mathtt{h}'$ and $\mathtt{h} \multimap \mathtt{h}, \mathtt{A}$, respectively.

Events $\mathtt{now} + \mathtt{t} \gg \mathtt{@Q}\,\{\,S\,\} \Rightarrow \mathtt{@Q}'$ define a statement S to be executed if, *after* t time units from the current execution, the contract is in the state Q. The expression t is either a non-negative number or a variable (a formal parameter or a field) or a variable plus a non-negative number. If the event is executed, the contract will transit to the state $\mathtt{Q}'$. The expression $\mathtt{now} + \mathtt{t}$ is called *time guard*. Functions and events are generically called *clauses*.

Expressions E include integer constant values v, the keyword now (that stands for a non-negative integer value), names of assets, fields and parameters generically ranged over by X, standard arithmetic and Boolean operations op (binary) and nop (unary). Booleans are encoded by integers in the usual way. Actually, *Stipula* has a richer set of values (reals, strings, time and asset values). For the time being, we stick to integer arithmetic, where deductive verification systems exhibit a high degree of automation. Other datatypes require corresponding support in automated theorem provers and SMT solvers: experimentation is needed to determine what is possible. We also want to keep the presentation simple.

The semantics of *Stipula* is defined operationally by means of a transition relation between *configurations* $\mathbb{C}, \mathbb{t}$, where $\mathbb{t}$ is a positive integer value representing the system's global clock and $\mathbb{C}$ is a tuple $\mathtt{C}(\mathbb{Q}\,,\,\ell\,,\,\Sigma\,,\,\Psi)$, where:

- C is the *Stipula* contract name;
- $\mathbb{Q}$ is the contract's current state: either _ (for no state) or a contract state Q;
- ℓ maps names (parties, fields, assets, function parameters) to values;
- Σ is the (possibly empty) residual of a function body or of an event handler, *i.e.* Σ is either _ or a term $S\ W \Rightarrow \mathtt{@Q}$;
- Ψ is a (possibly empty) multiset of *pending events* to be executed. We let Ψ be _ when there are no pending events, or

$$\mathbb{t}_1 \gg_{\mathtt{ev}_1} \mathtt{@Q}_1\,\{\,S_1\,\} \Rightarrow \mathtt{@Q}'_1 \quad | \cdots | \quad \mathbb{t}_n \gg_{\mathtt{ev}_n} \mathtt{@Q}_n\,\{\,S_n\,\} \Rightarrow \mathtt{@Q}'_n \ , \qquad (1)$$

[Function]

$$\frac{\begin{array}{c}\texttt{@Q A : f}(\overline{\texttt{y}})[\overline{\texttt{k}}]\,(E)\,\{\,S\,W\,\} \Rightarrow \texttt{@Q}' \in \texttt{C} \\ \Psi\ ,\ \mathbb{t} \nrightarrow \\ \ell(\texttt{A}) = A \quad \ell' = \ell[\overline{\texttt{y}} \mapsto \overline{u}, \overline{\texttt{k}} \mapsto \overline{v}] \quad [\![E]\!]_{\ell'} \neq 0\end{array}}{\texttt{C}(\texttt{Q}\,,\,\ell\,,\,_\,,\,\Psi)\ ,\ \mathbb{t} \xrightarrow{\texttt{A:f}(\overline{\texttt{u}})[\overline{\texttt{v}}]} \texttt{C}(\texttt{Q}\,,\,\ell'\,,\,\texttt{S W} \Rightarrow \texttt{@Q}'\,,\,\Psi)\ ,\ \mathbb{t}}$$

[State-Change]

$$\frac{[\![W\{^{\mathbb{t}}/_{\texttt{now}}\}]\!]_{\ell} = \Psi'}{\begin{array}{l}\texttt{C}(\texttt{Q}\,,\,\ell\,,\,_\,\texttt{W} \Rightarrow \texttt{@Q}'\,,\,\Psi)\ ,\ \mathbb{t} \\ \quad \longrightarrow \texttt{C}(\texttt{Q}'\,,\,\ell\,,\,_\,,\,\Psi' \mid \Psi)\ ,\ \mathbb{t}\end{array}}$$

[Event-Match]

$$\frac{\Psi = \mathbb{t} \gg_{\texttt{ev}_{\kappa}} \texttt{@Q}\ \{\ S\ \} \Rightarrow \texttt{@Q}' \mid \Psi'}{\texttt{C}(\texttt{Q}\,,\,\ell\,,\,_\,,\,\Psi)\ ,\ \mathbb{t} \xrightarrow{\texttt{ev}_{\kappa}} \texttt{C}(\texttt{Q}\,,\,\ell\,,\,\texttt{S} \Rightarrow \texttt{@Q}'\,,\,\Psi')\ ,\ \mathbb{t}}$$

[Tick]

$$\frac{\Psi\ ,\ \mathbb{t} \nrightarrow}{\begin{array}{l}\texttt{C}(\texttt{Q}\,,\,\ell\,,\,_\,,\,\Psi)\ ,\ \mathbb{t} \\ \quad \longrightarrow \texttt{C}(\texttt{Q}\,,\,\ell\,,\,_\,,\,\Psi \downarrow_{\mathbb{t}})\ ,\ \mathbb{t} + 1\end{array}}$$

Table 1. The control transition rules of *Stipula*

where $\mathbb{t}_i$ are the values of the expressions **now** + $\texttt{t}_i$ of the corresponding events. The indices $\texttt{ev}_i$ are introduced by rule [Function] when the body of a function is incorporated into the configuration and denotes a unique identifier statically assigned to the event.

The transition relation of *Stipula* is $\mathbb{C},\mathbb{t} \xrightarrow{\mu} \mathbb{C}',\mathbb{t}'$, where μ may be empty, or an initial agreement, or $A : \texttt{f}(\overline{u})[\overline{v}]$ (in case of function invocations) or an event. Its formal definition is given in Table 1 for the control transition rules, see [8] for the agreement and statement rules. The former rule set requires a specific JML/Java encoding, whereas the latter are translated in a standard way and are inessential for the understanding of this paper.

The formal definition of $\mathbb{C},\mathbb{t} \xrightarrow{\mu} \mathbb{C}',\mathbb{t}'$ uses the following auxiliary predicates and functions:

- $[\![E]\!]_{\ell}$ is a *partial function* returning the value of E in memory ℓ.
- $[\![W\{^{\mathbb{t}}/_{\texttt{now}}\}]\!]_{\ell}$ is the multiset of scheduled events obtained from the sequence W by replacing every now by $\mathbb{t}$ and evaluating every expression in the time guards.
- $\Psi\ ,\ \mathbb{t} \nrightarrow$ is a predicate that is *true* whenever Ψ is as in equation (1) and, for every $1 \leqslant i \leqslant n$, $\mathbb{t}_i \neq \mathbb{t}$. It is *false* otherwise.
- $\Psi \downarrow_{\mathbb{t}}$ removes from Ψ every event with time value less or equal to $\mathbb{t}$.

We explain the rules in Table 1. Rule [Function] defines invocations: the label specifies party A performing the invocation of a function named f with the actual parameters. The transition may occur provided (i) the contract is in the state Q, admitting invocations of f from A, (ii) the contract is *idle*, *i.e.* the contract has no statement to execute (presence of _ in the left configuration), (iii) the precondition E is satisfied, and no event can be triggered (premise $\Psi\ ,\ \mathbb{t} \nrightarrow$). In particular, the final constraint expresses that events *have precedence* on possible function invocations. For example, if a payment deadline is reached and, at the same time, the payment arrives, it will be refused in favour of the event managing the deadline. We use two notations for party identifiers: A and A. The former denotes the formal name used in the contract specification, whereas A refers to the concrete party name instantiated and fixed when the agreement is concluded.

Rule [State-Change] says that a contract changes state when the execution of the statement in the function body terminates. At this stage, the sequence of events W is added to the multiset of pending events once their time expressions have been evaluated, in particular `now` is replaced by the current value of the clock. Rule [Event-Match] specifies that event handlers may run provided there is no statement to perform and the time guard of the event has exactly the value of $\mathbb{t}$. Rule [Tick] defines the elapsing of time, which may happen when there is no statement to perform and no event can be triggered.

In this paper we use the state transition models of *Stipula* contracts, called the *underlying automata*. The states of these automata are those of the *Stipula* contract. The transitions correspond to clauses and are labelled either with the function name (the party name is always omitted, for simplicity's sake we assume that function names are pairwise different) or with a uniquely numbered event. By definition, these automata are *finite models*; Figures 2 and 3 show the two underlying automata of the contracts `Licence` and `Deposit`. These contracts are drawn from [11] with minor modifications, which are discussed below.

Example 1. Figure 2 defines the `License` contract regulating a licensing transaction between a `Licensor` and a `Licensee`, with a 30 days trial period and the possibility to purchase or decline the license. The field `code` stores the information for the trial period access, the asset `token` grants the licence. If the `Licensee` does not purchase the license before the trial period expires, the contract terminates and the `token` is automatically returned to the `Licensee`. The **`agreement`** in Line 4 allows the parties to agree on the license `cost`. Upon agreement, the first state is `@Init` where the `Licensor` may invoke `offer`, transferring a `token`

```
1  stipula License {
2      assets balance, token
3      fields cost, code
4      agreement (Licensor, Licensee)(cost) {
5          Licensor, Licensee : cost
6      } ⇒ @Init
7      @Init Licensor: offer(x)[n] {
8          n ⊸ token
9          x ⟶ code
10         now + 1 ≫ @Prop { token ⊸ Licensor } ⇒ @End
11     } ⇒ @Prop
12     @Prop Licensee: activate()[b] (b == cost) {
13         b ⊸ balance
14         code ⟶ Licensee ;
15         now + 30 ≫ @Trial {
16                 balance ⊸ Licensee
17                 token ⊸ Licensor
18                 } ⇒ @End
19     } ⇒ @Trial
20     @Trial Licensee: buy()[] {
21         balance ⊸ Licensor
22         token ⊸ Licensee
23     } ⇒ @End
24 }
```

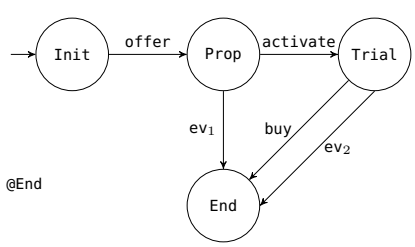

Fig. 2. The License contract in *Stipula* and its underlying automaton.

(representing the license) into escrow and a trial `code`. A scheduled `event` is si-

```
stipula Deposit {
    asset flour
    field cost_flour
    agreement (Client, Farm)(cost_flour) {
        Client, Farm : cost_flour
    } ⇒ @Start
    @Start Farm : begin()[h]{
        h ⊸ flour;
        now + 365 ≫ @Run { flour ⊸ Farm } ⇒ @End
    } ⇒ @Run
    @Run Farm : send()[k]{
        k ⊸ flour
    } ⇒ @Run
    @Run Client : buy()[w](w/cost_flour <= flour){
        (w/cost_flour) ⊸ flour, Client
        w ⊸ Farm
    } ⇒ @Run
}
```

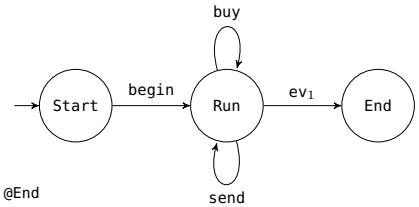

Fig. 3. The `Deposit` contract in *Stipula* and its underlying automaton

multaneously registered after 1 day to reclaim the token if the `Licensee` fails to start the trial period. The contract then moves to `Prop`. If no function is called in this state, then time is advanced, eventually triggering the event at line 10. In `Prop`, the `Licensee` can activate the trial by paying the `cost` (observe that `b` is an asset of the `Licensee`), which is transferred to the contract's `balance`: the contract acts as a notary for assets that are not finally disposed. Now the license code is revealed to the `Licensee` and another `event` is scheduled to handle the expiration of the trial period: if the license is not purchased before 30 days, the balance is refunded, the token returned to the `Licensor`. The license purchase is done by the `buy` function, which finalises the transaction by transferring the `balance` to the `Licensor` and assigning the `token` permanently to the `Licensee`. In the corresponding contract in [11], the start and end of the trial period are defined during the agreement, whereas here, for simplicity, they are hard-coded in the contract.

Example 2. Figure 3 defines the `Deposit` contract that models the interactions between a `Farm` and a `Client`. The `Farm` deposits flour, while the `Client` purchases and withdraws the corresponding amount at an agreed price. The contractual terms are enforced over a validity period of 365 days. The clause `send` allows the `Farm` to deposit flour into the contract's stock; the operation `h⊸flour` implements the deposit in the asset `flour`. Actual delivery occurs only with `buy()[w]`, where `(w/cost_flour)⊸flour, Client` transfers flour from the contract's stock to the `Client` (hence `w/cost_flour` must not be greater than `flour`) and `w⊸Farm` represents the payment to the `Farm`. In [11], the `Deposit` contract does not allow `buy` and `send` to start and end in the same state, as the underlying theory does not cover this case. Consequently, a more convoluted contract was required. In contrast, the definition of `Deposit` presented here is simpler, as it relies on a more expressive theoretical framework.

`License` and `Deposit` represent paradigmatic interaction patterns that frequently arise in legal contracts and that are clearly reflected in their underlying automata. The behavioural structure of `License` is *acyclic*: no state is revisited

during execution. In contrast, `Deposit` exhibits a *cyclic* pattern, as it includes functions that may be invoked repeatedly, leading to repeated traversals of certain states. It is important to observe that, while the semantics of *Stipula* admits potentially infinite cyclic behaviour (for example, `Deposit` allows an unbounded number of interactions in a fixed time window of 365 days), in practice only finitely many transactions can realistically occur. In Section 5, we use this observation to obtain bounded encoding of cycles in the translated contracts.

Definition 1 (Cyclic Contract, Separate Cycle). *A Stipula contract is* cyclic *if its underlying automaton has a* cycle*: a sequence* $\mathtt{Q}_1 \xrightarrow{\nu_1} \mathtt{Q}_2 \xrightarrow{\nu_2} \cdots \xrightarrow{\nu_{\ell-1}} \mathtt{Q}_\ell \xrightarrow{\nu_\ell} \mathtt{Q}_1$ *where* $\mathtt{Q}_1, \ldots, \mathtt{Q}_\ell$ *are pairwise different and* ν_i *are either function names or events. A contract has* separate cycles *if any two cycles either have no state in common or share their initial state only. A contract is* acyclic *if it has no cycle.*

The contract `License` is acyclic, while `Deposit` has separate cycles. In what follows, we first consider acyclic contracts and then turn to separate cyclic ones. This progression allows us to introduce the complexities of the translation into `JML`/`Java` in a gradual manner.

2.2 Deductive Verification with the KeY System

The KeY system [2, 4] is a deductive verification framework for `Java` that combines symbolic execution, invariant reasoning, and method contracts[3] on top of a calculus for an expressive program logic. The main use case of KeY is formal verification of Java programs annotated with specifications written in `JML`. It provides an interactive user environment, where one can construct correctness proofs of `Java` methods against their `JML` contracts. Method contracts typically include preconditions (**`requires`**), postconditions (**`ensures`**), frame conditions (**`assignable`**), class invariants, and auxiliary annotations such as loop invariants and assertions. Given a `JML`-annotated `Java` class, KeY translates the specifications into logical proof obligations and attempts to discharge them using its symbolic execution engine. The details of the verification process are irrelevant for the purpose of this paper: KeY is used as a *black box*, the input is a `JML`-annotated Java file that results from translation of a *Stipula* contract. In principle, *any* deductive verification system with a sufficient coverage of `JML`/`Java` can be used, for example, Krakatoa [9] or OpenJML [7], although we did not try that. In general, KeY is used in interactive or auto-active mode, however, for all case studies discussed below, the verification is *fully automatic*.

3 Translation Approach

To enable formal verification of a *Stipula* contract with KeY, we define a systematic translation into `Java` code annotated with `JML` specifications. As discussed

[3] Not to be confused with *Stipula* contracts. In this paper, both kinds of contract are featured, but it should be obvious from the context when we mean *Stipula* contracts and when `JML`/`Java` method contracts.

in [11], *Stipula* contracts are translated into classes where functions and events are modelled by stateful `Java` methods, assets and fields are modelled by suitable `Java` fields. The definition of the translation is given in Listing 1.

We begin by analysing the declaration component of the translation. As mentioned in Section 2.1, we assume all *Stipula* fields, assets, and auxiliary variables are of type `int` which preserves the operational semantics of the original contracts.[4] Verification involves a trade-off between precision and generality: parameters may be assigned concrete values, thereby analysing a specific execution scenario, or left symbolic to reason about all possible executions. The translation in Listing 1 adopts the former approach, targeting a single scenario. Accordingly, fields $\overline{\mathtt{x}}$ are initialised to constant values $\overline{u}$ (the values the fields have after the agreement), and formal parameters of functions are likewise instantiated to fixed values ($\overline{\mathtt{y_1}} = \overline{v_1}, \overline{\mathtt{k_1}} = \overline{v'_1}, \ldots, \overline{\mathtt{y_n}} = \overline{v_n}, \overline{\mathtt{k_n}} = \overline{v'_n}$).

```
TRANSLATE( stipula C { assets h̄ fields x̄ agreement( Ā ) { ··· } => @Q F ) ≝
  public final class C {
     public static int Ā, h̄, (A_h)^{A∈Ā,h∈h̄} ;
     public static int x̄ = ū ;
                    // ū are the values the fields x̄ have after the agreement
     public static int now = 0 ;
     public static int u_Q1, ..., u_Qm, w_Q1, ..., w_Qm ;
                    // auxiliary variables: Q1, ..., Qm are the states of C
     public static int[] DT = new int[N] ;
                    // DT is the dispatch table, N is the number of events in C
     public static int ȳ1 = v̄1, k̄1 = v̄'1, ..., ȳn = v̄n, k̄n = v̄'n ;
                    // formal parameters of functions and their initialization
     public static invariant ASSET_INV_C ;
     public static invariant 1 <= w_Q1 <= max_Q1 && ··· && 1 <= w_Qm <= max_Qm ;
        // max_Q1,...,max_Qm are the number of functions starting at Q1,...,Qm
     public static invariant 0 <= u_Q1 <= max_tv && ··· && 0 <= u_Qm <= max_tv ;
                    // max_tv is a constant, see Section 4
     TRANSLATE(F)

     /*@ public normal_behaviour
       @ requires h̄ == 0 ; DT[0] == -1 && ... && DT[N-1] = -1 ;
         // assets of C are initially 0, DT is empty (encoded by value -1)
       @ requires OTHER PRE-CONDITIONS ;
       @ ensures THE POST-CONDITION ;       // the property we want to prove
       @*/

     public final static void behaviour(){  GEN_C(Q) }
}
```

Listing 1. The TRANSLATE function for a *Stipula* contract

The translation introduces auxiliary variables: The dispatch table `DT` is an array of size `N`, where `N` is the number of events in the contract. For each contract

[4] *Stipula* itself is type-free: concrete types (e.g., integers, doubles, strings, Booleans) are inferred statically and at run time [8].

state `Q1`, ..., `Qm`, the variable `u_Qi` tracks time progression and ranges over non-negative values bounded by `max_tv`, the largest constant (plus one) appearing in time expressions (in principle, a single variable would suffice). For each state `Qi`, the variable `w_Qi` counts function invocations and ranges between 0 and `max_Qi` (the number of functions whose initial state is `Qi`).

Assets satisfy a non-duplication property: *the total amount of a given asset in the system* – that is, in the contract and in all the parties – *must remain constant*. To enforce this property, we introduce a field A_h for every party A and every asset `h` in class `C`. Moreover, we enforce a static invariant $\text{ASSET_INV}_{\texttt{C}}$ that holds for every method in `C` as follows:

$$
\begin{aligned}
&\text{ASSET_INV}_{\texttt{C}} \stackrel{\textsf{def}}{=} \\
&\quad h_1 \texttt{ + } A_1_h_1 \texttt{ + } \cdots \texttt{ + } A_k_h_1 \texttt{ == } \texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(}h_1\texttt{) + }\texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(}A_1_h_1\texttt{) + } \cdots \texttt{ + }\texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(}A_k_h_1\texttt{)} \\
&\quad \texttt{\&\&} \cdots \texttt{\&\&} \\
&\quad h_r \texttt{ + } A_1_h_r \texttt{ + } \cdots \texttt{ + } A_k_h_r \texttt{ == } \texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(}h_r\texttt{) + }\texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(}A_1_h_r\texttt{) + } \cdots \texttt{ + }\texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(}A_k_h_r\texttt{)}
\end{aligned}
$$

In `JML` the **`old`** construct is used to refer to the values of fields *prior to method execution*. This allows us to precisely relate the post-state of a method to its pre-state, which is essential for enforcing the non-duplication property. For example, $\text{ASSET_INV}_{\textsf{License}}$, the invariant of the License contract is

```
   balance + Licensor_balance + Licensee_balance ==
       \old(balance) + \old(Licensor_balance) + \old(Licensee_balance)
&& token + Licensor_token + Licensee_token ==
       \old(token) + \old(Licensor_token) + \old(Licensee_token)
```

When assets are transferred directly from one party to another without being stored in a contract's asset (for example, the `buy` function in the `Deposit` contract), we also add the invariant

$$\texttt{A}_1 \texttt{ + } \cdots \texttt{ + } \texttt{A}_k \texttt{ = } \texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(A}_1\texttt{) + } \cdots \texttt{ + }\texttt{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{\textbf{old}}}}}}}}}}}\texttt{(A}_k\texttt{)}$$

modeling that, in these cases, assets are transferred directly to the party's identifier (which was introduced for this purpose).[5]

Functions and events are modeled as static `Java` methods annotated with `JML` specifications consisting of three clauses. Consider a *Stipula* function declared as $\texttt{@Q A : f(}\overline{\texttt{y}}\texttt{)[}\overline{\texttt{k}}\texttt{]}\,(E)\,\{\,S\,W\,\} \Rightarrow \texttt{@Q}'$. Its translation, defined in Listing 2, includes:

- A **`requires`** clause expressing that guard E that must hold in the initial state for the function to execute.
- An **`ensures`** clause specifying the resulting state transition, including field updates and asset transfers. This clause is defined via the strongest postcondition $\text{STRONG_POSTCOND}(E, S)$ computed from the pre-state (see [4] for details about strong post-conditions).
- An **`assignable`** clause listing the variables that may be modified, determined by a simple static "writes-to" analysis $\text{LHSVAR}_{\texttt{A}}(S)$.

[5] In [11], assets are classified as *divisible* (*e.g.*, digital currencies) or *indivisible* (*e.g.*, non-fungible tokens), thus generating appropriate invariants to enforce the intended asset semantics. Here, for simplicity, we assume that every asset is divisible.

The body of each method corresponding to a *Stipula* function invoked by a party `A`, as well as the bodies of events declared within that function, are generated via the function $\textsc{Stipula2Java}_{\mathtt{A}}(S)$. This function maps a *Stipula* statement S to a sequence of `Java` statements, yielding a compositional encoding of the function's operational semantics into executable `Java` code.

```
TRANSLATE( @Q A : f(y̅)[k̅] (E) { S W } ⇒ @Q′ ) ≝
 /*@ public normal_behaviour
   @ requires E ;
   @ ensures  STRONG_POSTCOND(E, S) && ENSUREDT_f ;
   @ assignable LHSVAR_A(S), DT ;
   @*/
 public final static void f(int y̅, int k̅) {
     STIPULA2JAVA_A(S)
     UPDATEDT(W)
}
 /*@ public normal_behaviour        // for every now + t_i ≫ @Q_i { S_i } ⇒ @Q′_i ∈ W
   @ ensures  STRONG_POSTCOND(true, S_i) ;
   @ assignable LHSVAR_A(S_i) ;
   @*/
 public final static void event_i() {
     STIPULA2JAVA_A(S_i)
}
```

Listing 2. The TRANSLATE function for *Stipula* functions and events

The definition of $\textsc{Stipula2Java}_{\mathtt{A}}(S)$ follows a well-established and standard formulation. For readability, the definition is relegated to the Addendum A at the end of the paper.

The discussion about the function $\textsc{gen}_{\mathtt{C}}(\mathtt{Q})$ (in Listing 1) and the functions $\textsc{updatedt}(W)$ and $\textsc{ensuredt}_{\mathtt{f}}$ (in Listing 2) are deferred to the next section, as they rely on the dispatch table mechanism.

4 The Dispatch Table and Generation of Execution Paths

To complete the translation, we encode the behaviour of a *Stipula* contract. We use three key mechanisms:

1. Event declarations are collected into a *dispatch table*, recording enabled events and their execution times.
2. *Symbolic execution* implicitly enumerates admissible scenarios via non-deterministic function and event selection;
3. Time progression in a state `Q` is bounded by a constant `max_tv`, defined as the maximum constant (plus one) occurring in time expressions, and tracked through the symbolic variable `u_Q` $\in$ [0..`max_tv`]. Advancing time beyond this bound is unnecessary, as no pending event can be executed past that deadline unless generated by a subsequent function invocation.

The dispatch table, introduced in Listing 1, is an array `DT` of size `N`, where `N` is the number of events in the contract. The i-th entry corresponds to the i-th event **now** + $\mathtt{t}_i \gg$ @$\mathtt{Q}_i$ $\{S_i\}$ ⇒ @$\mathtt{Q}'_i$. The value `DT[i]` is `-1` if the event has not been created; otherwise, it stores the scheduled execution time `now` + $\mathtt{t}_i$, where `now` denotes the time at which the event was generated.

Events declared in a function are inserted in Listing 2 into the dispatch table by UPDATEDT(W) . If a function `f` declares events indexed by $\mathtt{ev}_{\jmath_1}, \ldots, \mathtt{ev}_{\jmath_{N_\mathtt{f}}}$ with time expressions `now` + $\mathtt{t}_{\jmath_i}$, then UPDATEDT(W) assigns the corresponding computed times to the entries `DT[`$\mathtt{ev}_{\jmath_i}$`]`. In other words, it updates exactly those positions associated with the events generated by `f`. Entries of `DT` not corresponding to events in W remain unchanged, while the relevant entries are updated to `now` + $\mathtt{t}_{\jmath_i}$ using the current value of `now`. The formalisation of this behaviour in `JML` is done by the predicate ENSUREDT$_\mathtt{f}$. Thus, the specification guarantees that precisely the events declared in `f` are scheduled, relative to the current time `now`, and that no other dispatch table entries are affected. (The formal definition of UPDATEDT(W) and ENSUREDT$_\mathtt{f}$ is omitted, because it is straightforward.)

To obtain the possible behaviour of a *Stipula* contract, the dispatch table will be queried to identify one of the events that can be enabled next. To this end, we introduce the auxiliary `Java` method

```
int minTimeEntry({ev1,..., evm}) .
```

The method returns the index in the dispatch table entry corresponding to an event in $\{\mathtt{ev}_1, \ldots, \mathtt{ev}_m\}$ whose associated time value is minimal, provided that such an event exists; otherwise, it returns `-1`. The method is non-deterministic, as multiple events may share the same minimal time value in the dispatch table. In this case, any one of these events may be selected, thereby allowing the symbolic execution to explore all admissible scheduling choices. The implementation of the method `minTimeEntry()` is omitted. Instead, we provide its `JML` specification that defines the essential semantic properties required for verification, which is sufficient for KeY to reason symbolically about the method's behaviour:

```
/*@ public normal_behaviour // specification of int minTimeEntry({ev1,..., evm})
  @ ensures ( \exists i; 0 <= i < N; (i ∈ {ev1, ..., evm} && DT[i] >= now) )
            ⇒ \result ∈ {ev1, ..., evm} && DT[\result] >= now &&
              \forall j; 0 <= j < N; (j ∈ {ev1, ..., evm} && DT[j] >= now)
                                          ⇒ DT[j] >= DT[\result] ;
  @ ensures ( \forall i; 0 <= i < N; i ∈ {ev1, ..., evm} ⇒ DT[i] < now)
            ⇒ \result == -1;
  @ assignable \nothing;
  @*/
```

The first **ensures** clause says that when there is an index in $\{\mathtt{ev}_1, \ldots, \mathtt{ev}_m\}$ such that its time value in `DT` is greater or equal to `now`, then such an index is returned. There can be more than one such index. This causes symbolic execution to branch over all indices `\result` such that `DT[\result] >= now`. The second **ensures** clause covers the case when no entry is found. In this case `\result` is `-1`.

We can now complete the definition of TRANSLATE (Listing 2) by specifying the function GEN$_\mathtt{C}$(`Q`) (Listing 3). Assume that in state `Q` functions $\mathtt{f}_1, \ldots, \mathtt{f}_n$ can

```
GEN_C(Q) ≝
    int entry = minTimeEntry({ev_1,..., ev_m}) ;
    if ((entry == -1) || (DT[entry] > now && w_Q > 0)) { // call a function
       if (entry == -1) now = now + u_Q ; else now = min(DT[entry]-1, now+u_Q) ;
       if (w_Q == 1) { f_1(y_1, k_1) ; GEN_C(Q_1) }
       else if (w_Q == 2) { f_2(y_2, k_2) ; GEN_C(Q_2) }
       ...
       else { f_n(y_n, k_n) ; GEN_C(Q_n) } // max_Q == n, see Listing 1
    } else { // trigger an event
       now = DT[entry] ;
       if (entry == 1) { event_1() ; DT[entry] = -1 ; GEN_C(Q'_1) }
       else if (entry == 2) { event_2() ; DT[entry] = -1 ; GEN_C(Q'_2) }
       ...
       else { event_m() ; DT[entry] = -1 ; GEN_C(Q'_m) } // entry == m
    }
```

Listing 3. The function $\text{GEN}_{\mathtt{C}}(\mathtt{Q})$; the $\mathtt{f}_1, \ldots, \mathtt{f}_n$ and $\mathtt{event_1}, \ldots, \mathtt{event_m}$ are the functions and events that start in $\mathtt{Q}$ and end in $\mathtt{Q}_1, \ldots, \mathtt{Q}_n$ and $\mathtt{Q}'_1, \ldots, \mathtt{Q}'_m$, respectively.

be called and events $\mathtt{event}_1, \ldots, \mathtt{event}_m$ can be triggered. The *Stipula* semantics is non-deterministic. To model this, we use two symbolic variables that were declared in Listing 1: $\mathtt{w_Q} \in [\mathtt{0..max_Q}]$, for selecting a function in the state `Q`, and $\mathtt{u_Q} \in [0..\mathtt{max_tv}]$, for controlling time progression. The behaviour is as follows:

1. *Enabled events at current time.* If `minTimeEntry(...)` returns an index whose time equals `now`, the corresponding event *must* execute (events preempt functions). After execution, the event is removed from `DT`. (This removal is actually unnecessary since the automaton is acyclic, it cannot occur again.)
2. *Pending future events.* If one of the earliest scheduled events occurs at a time `entry > now`, then two cases arise: either time advances, but not until `entry`, i.e. at most to `min(DT[entry]-1, now+u_Q)` and a function is executed (`w_Q > 0`); otherwise, time advances *exactly* to `t` and the event is executed (`w_Q = 0`).
3. *No pending events.* If no event from `Q` remains to be scheduled, a function must eventually execute. Time progression is bound by `max_tv`, defined as the largest constant (plus one) appearing in time expressions. Assigning `u_Q ∈ [0..max_tv]` ensures a finite, yet sound exploration of all relevant execution scenarios.

Now that the translation is complete, it becomes possible to define and verify meaningful properties for *Stipula* contracts translated to `JML/Java` by filling in the slots for `OTHER PRE-CONDITION` and `THE POST-CONDITION`, respectively, in Listing 1. For example, one can prove with KeY the following property of the `License` contract:

```
requires Licensor_balance >= 0 && Licensee_balance > 0 &&
         Licensor_token > 0    && Licensee_token == 0 ;
```

```
ensures  (Licensor_balance == \old(Licensor_balance)+cost && Licensor_token == 0
          && Licensee_balance == \old(Licensee_balance)-cost
          && Licensee_token == \old(Licensor_token)
         ) || (
          Licensor_balance == \old(Licensor_balance) &&
          Licensor_token == \old(Licensor_token) &&
          Licensee_balance == \old(Licensee_balance) &&
          Licensee_token == \old(Licensee_token)
         ) ;
```

The contract expresses that, upon termination, either the licensing transaction succeeded (the `Licensee` purchased the license and the `Licensor` obtained the money) or the `Licensee` is withdrawn and no `token` was delivered.

Theorem 1 (Soundness). *Let* $\mathtt{C}$ *be an acyclic* ***Stipula*** *contract and let*

$$\mathbb{C},\mathtt{O} \xrightarrow{(\overline{A},\,\overline{A_i}:\overline{v_i}^{\,i\in 1..\ell})}{}^{*}\ \mathbb{C}_1,\mathbb{t}_1 \xrightarrow{\mu_1}{}^{*}\ \mathbb{C}_2,\mathbb{t}_2 \xrightarrow{\mu_2}{}^{*}\ \cdots \xrightarrow{\mu_{n+1}}{}^{*}\ \mathbb{C}_{n+1},\mathbb{t}_{n+1}$$

where $\mathtt{Q}_i$ *are the states of* $\mathbb{C}_i$*, with* $i \in 1..n+1$*,* $\mathtt{Q}_{n+1}$ *is the final state of the underlying automaton, and* μ_i *are either function invocations* $A_i{:}\mathtt{f}_i(\overline{v_i})[\overline{v_i'}]$ *or events* $\mathtt{ev}_i$*. Then the above computation may be traced in the code of* $\textsc{gen}_{\mathtt{C}}(\cdot)$ *as follows. Consider* $\mathbb{C}_i,\mathbb{t}_i \xrightarrow{\mu_i}{}^{*}\ \mathbb{C}_{i+1},\mathbb{t}_{i+1}$*; if, in the code of* $\textsc{gen}_{\mathtt{C}}(\mathtt{Q}_i)$*,*

1. ***now*** $= \mathbb{t}_i$ *if* μ_i *is an event;* ***now*** $\leqslant \mathbb{t}_i$ *if* μ_i *is a function;*
2. *if the multiset of pending events in* $\mathbb{C}_i$ *is*

$$\mathbb{t}_{\jmath_1} \gg_{\mathtt{ev}_{\jmath_1}} \mathtt{@Q}_{\jmath_1}\,\{\,S_{\jmath_1}\,\} \Rightarrow \mathtt{@Q}'_{\jmath_1} \mid \cdots \mid \mathbb{t}_{\jmath_\kappa} \gg_{\mathtt{ev}_{\jmath_k}} \mathtt{@Q}_{\jmath_k}\,\{\,S_{\jmath_k}\,\} \Rightarrow \mathtt{@Q}'_{\jmath_k}$$

and the dispatch table $\mathtt{DT}[\mathtt{ev}_{\jmath_i}] = \mathbb{t}_{\jmath_i}$*, with* $i \in 1..k$*.*

Then, whenever $\mu_i = A_i{:}\mathtt{f}_i(\overline{v_i})[\overline{v_i'}]$ *it is possible to execute the method* $\mathtt{f}_i(\overline{v_i},\overline{v_i'})$ *and if* $\mu_i = \mathtt{ev}_{\jmath_i}$*, it is possible to execute method* $\mathtt{event_}\jmath_i(\ )$ *and obtain a state where (1.) and (2.) hold for* $\mathbb{C}_{i+1}$ *,* $\mathbb{t}_{i+1}$*.*

Therefore, every computation of an acyclic ***Stipula*** contract is faithfully represented within the `Java` code generated by our translation (the proof is omitted, it is technical, but not difficult). Consequently, when the program is analysed with KeY, each legally admissible behaviour of the original contract is systematically explored. This guarantees that the verification process is sound with respect to the contract's operational semantics in the acyclic setting, as no admissible execution is omitted. We further conjecture that the translation is complete in the acyclic case, in the sense that every behaviour of the generated `Java` program corresponds to a computation that is admissible in the original ***Stipula*** contract. Establishing such a result formally, however, would require a formalisation of `Java` semantics, which lies beyond the scope of the present paper.

5 Cyclic Behaviour and its Translation

The technique of Sections 3 and 4 extends to cyclic contracts with separate cycles such as `Deposit`. Here we examine the additional challenges posed by cycles and clarify the resulting loss of precision.

In `Deposit`, the `Farm` and the `Client` may repeatedly invoke `send` and `buy`, forming a cycle in the contract underlying automaton (Figure 3). The cycle is exited through a dedicated event executed at most once. This reflects a common contractual pattern: (*i*) cycles have a unique entry/exit state and contain only function invocations; (*ii*) cycles are not nested; (*iii*) events occur outside cycles and typically enforce their termination. As a consequence, each event is executed at most once, even in the presence of cyclic behaviour.

This observation allows us to reuse the dispatch table mechanism developed for acyclic contracts, since the set of events remains finite. However, events generated inside cycles require special care: although they share the same index, different iterations may assign them different time values. To handle this, we assign such events an *abstract* clock value, encoded as `-2`. Rather than reasoning about their precise scheduling time, we only record that they have been generated. We illustrate the resulting loss of precision with an example. Consider the cyclic *Stipula* function declaration:

```
@Q f()[] {
   now + 1 ≫ @Q {} ⇒ @Q1
   now + 2 ≫ @Q {} ⇒ @Q2
} ⇒ @Q
```

According to the *Stipula* semantics, the second event is never triggered, because the first one always precedes it, so state `@Q2` is unreachable. But since the events occur in a cycle, both are given the abstract clock value `-2`, enabling an infeasible execution in the `JML`/`Java` model that triggers the second event and reaches `@Q2`.

For modeling cycles in `JML` we refine $\textsc{updatedt}(W)$ and distinguish whether the generating function of an event lies inside or outside a cycle. If it belongs to a cycle, the events in W are inserted into the dispatch table with the abstract time value `-2`, instead of a concrete timestamp. Accordingly, the predicate $\textsc{ensuredt}_{\mathtt{f}}$ is adapted to enforce that *exactly* the events generated by `f` are assigned the value `-2`, while all other entries of `DT` remain unchanged. The structure of the specification remains the same as in the acyclic case, differing only in the assigned time value. The remaining definitions in Section 4 are omitted, because they are straightforward.

The crucial adaptation concerns the extension of $\textsc{gen}_{\mathtt{C}}(\mathtt{Q})$ (Listing 3) to states within cycles. For such states, we use the notation $\textsc{gen}_{\mathtt{C}}^{\mathtt{Q},\mathtt{count}}(\mathtt{Q}')$, where `Q` denotes the entry/exit state of the cycle and `count` is the iteration counter introduced by the translation (see Listing 4). Each cycle is translated into a dedicated `Java` **`while`** loop whose body encodes all possible traversals of the cycle. A counter tracks the number of iterations, bounded by a cycle-specific constant $\mathtt{MAX_ITE}_{\mathtt{Q}}$. This bound becomes a parameter of the `behaviour` method in Listing 1 with pre-

condition **requires** $\mathtt{MAX_ITE_Q} \geqslant 0$ and provides a natural abstraction: although *Stipula* allows unbounded cyclic behaviour (*e.g.*, in `Deposit`), only finitely many interactions occur in practice. Finally, the time-progress variable `u_Q` used in the acyclic case is omitted for cyclic states, since events generated within cycles carry an undetermined time value and do not require explicit clock advancement.

From a deductive verification perspective, finding suitable loop invariants is a very difficult task in formal program verification and usually not amenable to automation. For this reason our translation technique leaves the supply of post condition-specific invariants to the verification engineer. For example, in the case of `Deposit`, using the following invariant (v_s is the actual value of the parameters `h` and `k` in `begin` and `send`; v_b is the value of `w` in `buy`) it is possible to derive in KeY the property that all the flour acquired by the `Client` has been paid to the `Farm` at agreed cost.

```
loop_invariant (Client_flour == \old(Client_flour) && flour == \old(flour)+v_s) ||
               (Client_flour == \old(Client_flour)+v_b/cost_flour &&
                Farm == \old(Farm)+v_b) ;
```

Another limitation of our current encoding is that function parameters are treated as symbolic constants throughout a loop, meaning all iterations are analysed under a single symbolic instantiation (e.g., v_s and v_b). While *Stipula* allows parameters to vary between iterations, verifying this non-determinism would require loop invariants over sequences of iteration-dependent values, which tools like KeY cannot currently handle automatically. Supporting such cases is theoretically possible with user-supplied invariants and interactive proofs, but will further reduce the automation of our approach.

For the cyclic case, we can prove a soundness theorem for the contracts analysed here, assuming constraints on repeated function calls with identical arguments and on the number of iterations.

Theorem 2 (Soundness). *Let* `C` *be a* *Stipula* *contract with separate cycles and events that are outside cycles. Let*

$$\mathbb{C},\mathtt{0} \xrightarrow{(\overline{A},\,\overline{A_i}:\overline{u_i}\;^{i\in 1..\ell})}{}^{*}\; \mathbb{C}_1,\mathbb{t}_1 \xrightarrow{\mu_1}{}^{*}\; \mathbb{C}_2,\mathbb{t}_2 \xrightarrow{\mu_2}{}^{*}\; \cdots \xrightarrow{\mu_{n+1}}{}^{*}\; \mathbb{C}_{n+1},\mathbb{t}_{n+1}$$

where $\mathtt{Q}_i$ *are the states of* $\mathbb{C}_i$*, with* $i \in 1..n+1$*, and* μ_i *are either function invocations* $A_i{:}\mathtt{f}_i(\overline{v_i})[\overline{v'_i}]$ *or events* $\mathtt{ev}_i$*. Additionally, if* μ_i *and* μ_j *are invocations of the same function then* $\mu_i = \mu_j$ *(the invocations have the same actual values). Then, the above computation may be traced in the code of* $\textsc{gen}_{\mathtt{C}}(\cdot)$ *as follows. Consider* $\mathbb{C}_i,\mathbb{t}_i \xrightarrow{\mu_i}{}^{*}\; \mathbb{C}_{i+1},\mathbb{t}_{i+1}$ *and assume that, in the code of* $\textsc{gen}_{\mathtt{C}}(\mathtt{Q}_i)$ *or of* $\textsc{gen}_{\mathtt{C}}^{\mathtt{Q},\mathtt{count}}(\mathtt{Q}_i)$ *(when* $\mathtt{Q}_i$ *is part of a cycle),*

1. ***now*** $= \mathbb{t}_i$ *if* μ_i *is an event not generated inside a cycle;* ***now*** $\leqslant \mathbb{t}_i$*, otherwise;*
2. *if the multiset of pending events in* $\mathbb{C}_i$ *is*

$$\mathbb{t}_{\jmath_1} \gg_{\mathtt{ev}_{\jmath_1}} \mathtt{@Q}_{\jmath_1}\,\{\,S_{\jmath_1}\,\} \Rightarrow \mathtt{@Q}'_{\jmath_1} \mid \cdots \mid \mathbb{t}_{\jmath_\kappa} \gg_{\mathtt{ev}_{\jmath_k}} \mathtt{@Q}_{\jmath_k}\,\{\,S_{\jmath_k}\,\} \Rightarrow \mathtt{@Q}'_{\jmath_k}$$

and the dispatch table is such that $\mathtt{DT}[\mathtt{ev}_{\jmath_i}] \in \{\mathbb{t}_{\jmath_i}, \mathtt{-2}\}$*, with* $i \in 1..k$*.*

There are two cases:
(1) Q is the entry/exit state of a cycle; $\mathtt{f}_1, \ldots, \mathtt{f}_n$ and $\mathtt{ev}_{j1}, \ldots, \mathtt{ev}_{jm}$ are functions with initial state Q and final states $\mathtt{Q}_1, \ldots, \mathtt{Q}_n$ and $\mathtt{Q}'_1, \ldots, \mathtt{Q}'_m$, respectively.

```
GEN_C(Q) ≝
  int entry = minTimeEntry({ev_j1, ··· , ev_jm}) ;
  if ((entry == -1) || (DT[entry] > now && w_Q > 0)) {
    int count = 0 ;
    /*@ loop_invariant 0 <= count <= MAX_ITE_Q ; // bounds
      @ loop_invariant ... ; // assets, fields, and DT invariants
      @ decreases MAX_ITE_Q - count ; // termination expression
      @*/
    while (count < MAX_ITE_Q) {
      if (w_Q == 1) { f_1(x̄_1, k̄_1) ; if (Q != Q_1) GEN_C^{Q,count}(Q_1) }
      else if (w_Q == 2) { f_2(x̄_2, k̄_2) ; if (Q != Q_2) GEN_C^{Q,count}(Q_2) }
      ...
      else { f_n(x̄_n, k̄_n) ; if (Q != Q_n) GEN_C^{Q,count}(Q_n) } // max_Q == n
      count = count + 1 ;
    }
    entry = minTimeEntry({ev_j1,···,ev_jm}) ; // we are back in state Q
    if (DT[entry] > now || DT[entry] == -2) {
      if (DT[entry] > now) now = DT[entry] ;
      if (entry == ev_j1) { event_j1() ; GEN_C(Q'_1) }
      else if (entry == ev_j2) { event_j2(); GEN_C(Q'_2) }
      ...
      else { event_jm() ; GEN_C(Q'_m) } // entry == ev_jm
    }
  } else { if (DT[entry] > now) now = DT[entry] ;
    if (entry == ev_j1) { event_j1() ; GEN_C(Q'_1) }
    else if (entry == ev_j2) { event_j2(); GEN_C(Q'_2) }
    ...
    else { event_jm() ; GEN_C(Q'_m) } // entry == ev_jm
  }
```

(2) Q′ is a state of a cycle that is *not* the entry/exit state. Let $\mathtt{f}_1, \ldots, \mathtt{f}_\ell$ be the functions whose final states $\mathtt{Q}_1, \ldots, \mathtt{Q}_\ell$ are in the cycle.

```
GEN_C^{Q,count}(Q') ≝ if (w_Q' == 1) { f_1(x̄_1, k̄_1) ; if (Q != Q_1) GEN_C^{Q,count}(Q_1) }
                      else if (w_Q' == 2) { f_2(x̄_2, k̄_2) ; if (Q != Q_2) GEN_C^{Q,count}(Q_2) }
                      ...
                      else { f_ℓ(x̄_ℓ, k̄_ℓ) ; if (Q != Q_ℓ) GEN_C^{Q,count}(Q_ℓ) }
```

Listing 4. The function $\mathrm{GEN}_{\mathtt{C}}^{\mathtt{Q},\mathtt{count}}(\mathtt{Q}')$.

Then, if $\mu_i = A_i{:}\mathtt{f}_i(\overline{v_i})[\overline{v'_i}]$ *it is possible to execute the method* $\mathtt{f}_i(\overline{v_i}, \overline{v'_i})$ *and if* $\mu_i = \mathtt{ev}_{\jmath_i}$*, it is possible to execute method* $\mathtt{event_}\jmath_i(\ )$ *and obtain a state where (1.) and (2.) hold for* $\mathbb{C}_{i+1}$ *,* $\mathbb{t}_{i+1}$*.*

This result crucially depends on the values of $\mathtt{MAX_ITE_Q}$. If reliable upper bounds on the number of possible iterations are known, the analysis carried out with KeY is guaranteed to explore all the legally admissible behaviour of the contract. However, in contrast to the acyclic setting, completeness can no longer be claimed. As discussed at the beginning of the section, the introduction of over-approximations, specifically, assigning the abstract time value `-2` to events generated inside cycles, may lead the translated `Java` program to exhibit behaviour not admissible by the semantics of *Stipula*. As a consequence, the verification performed by KeY may encounter spurious executions, potentially preventing the proof of properties that do hold in the original contract.

6 Related Work

Legal modeling frameworks like Ergo [19], OpenLaw [18], Lexon [14], and Accord [1] embed contracts into broader systems, but they lack a precise formal semantics. While Catala [15] formalizes legislative logic and has a runtime environment, it does not address contract verification. In contrast, *Stipula* offers an operational model with explicit permissions, assets, and timed clauses and has a comprehensive suite of tools, including a visual code editor for intuitive contract authoring, an interpreter for automatic execution, and a number of analyzers for verifying several legally relevant properties (unreachability of clauses, absence of frozen assets, etc.) [13].

Closer to our work, prior efforts explored translation-based verification: OCL-to-`Java` with `JML` [12], and Circus-to-`Java` for formal reasoning [10]. Our approach applies this paradigm to legal contracts, preserving their normative and temporal semantics.

More recent work models blockchain smart contracts (by hand) in a modeling language designed for verification which is then automatically translated to EVM bytecode [20], for example, Dafny-to-EVM [6] or Why3-to-EVM [16]. This is the *reverse* direction of our approach. While it makes verification easier, *validation* becomes harder.

While the current *Stipula* runtime is realized by compilation to `Java` [21], the language is implementation-agnostic and could be compiled to blockchain smart contracts like `Solidity` [20] or `Obsidian` [17]. This path is promising, as our verification approach using KeY [2] can build on established work that already applies the system to blockchain platforms like Hyperledger Fabric [5] and `Solidity` [3] by embedding them into `Java`.

7 Conclusion

The formal modeling and verification of digital contracts is gaining momentum at the intersection of legal informatics, programming languages, and formal meth-

Restriction	Motivation / How to Lift
Fields, assets and variables are of type integer, see Section 2.1	Increase degree of automation; can be lifted by dedicated automated reasoners
Cycles in contracts are separate and event-free, see Sections 2.1 and 5	General cycles are rare in legal contracts and significantly complicate the translation, because they cannot be modeled by loops
Events generated in cycles get abstract clock value, see Section 5	Precise treatment requires dynamic and unbounded extension of dispatch table
Cycle iterations are analysed under a single symbolic instantiation of the formal parameters, see Section 5	Requires more complex loop invariants that might be hard to find

Table 2. Restrictions to *Stipula* and motivations

ods. This work contributes by connecting the contract language *Stipula* with a deductive verification system (KeY) via a translation to `JML`-annotated `Java`.

We focus on two recurring interaction patterns in legal contracts: acyclic behaviour and non-nested cyclic behaviour consisting only of function calls. We also impose other technical restrictions which might be lifted in the future. Maximising automation has been our guiding principle. In the acyclic case, verification is fully automatic once the contract is specified. For cyclic behaviour, where stronger reasoning is needed, we aim to minimise additional annotations and proof guidance to preserve usability and scalability. From a theoretical perspective, the most challenging problem is the precise handling of clock values in events originating from cycles. It would require a generalisation of dispatch tables that precisely describes event generation for an *unbounded* number of iterations. A systematic, experimentally driven investigation of the trade-offs between coverage and precision constitutes an important direction for future work. Table 2 summarises the current restrictions and to which extent they could be dropped.

In general, a systematic empirical evaluation, especially for large and structurally complex cyclic contracts remains to be done. Preliminary results are promising but concern an earlier version of the translation without dispatch tables [11]. A comprehensive assessment of the current framework remains essential.

A Addendum: The Function Stipula2Java

The definition of $\textsc{Stipula2Java}_{\mathtt{A}}(S)$ follows.

- $\textsc{Stipula2Java}_{\mathtt{A}}(_) \stackrel{\text{def}}{=} \varepsilon$, where ε is the empty string.
- $\textsc{Stipula2Java}_{\mathtt{A}}(E \rightarrow \mathtt{x}\ \ S) \stackrel{\text{def}}{=} \mathtt{x} = E\ \mathtt{;}\ \textsc{Stipula2Java}_{\mathtt{A}}(S)$.
- $\textsc{Stipula2Java}_{\mathtt{A}}(E \rightarrow \mathtt{B}\ \ S) \stackrel{\text{def}}{=} \textsc{Stipula2Java}_{\mathtt{A}}(S)$. The value of E is irrelevant in our examples. If the value of E is relevant, we declare an *ad hoc* field, say `B_transmitted`, for storing the value.
- $\textsc{Stipula2Java}_{\mathtt{A}}(E \multimap \mathtt{h}, \mathtt{h}'\ \ S)$. There are two cases: (i) both $\mathtt{h}$ and $\mathtt{h}'$ are assets of the contract or

(*ii*) `h` is a formal parameter and `h'` is an asset of the contract. The code for case (*i*) and (*ii*) is (`tmp` is a fresh variable):

```
int tmp = E ;
//@ assert h-tmp >= 0 ;
h = h-tmp ; h' = h'+tmp ;
Stipula2Java_A(S)
```

Case *(i)*

```
int tmp = E ;
//@ assert h-tmp >= 0 && A_h'-tmp >= 0 ;
h = h-tmp ; A_h' = A_h'-tmp ;
h' = h'+tmp ;
Stipula2Java_A(S)
```

Case *(ii)*

According to the `JML` **`assert`** semantics, verification can only succeed provided that `h-tmp >= 0`, which is exactly what the *Stipula* semantics requires. In addition, in case (*ii*), the formal parameter `h` represents (part of) the asset `A_h'` that the caller `A` transfers to the contract upon invocation. Consequently, the decrease in `h` must be reflected in the caller's asset rather than in the contract state. This semantic correspondence motivates the assignment `A_h' = A_h'-tmp;` which correctly accounts for the transfer of the asset from the caller to the contract.

– $\textsc{Stipula2Java}_{\mathtt{A}}(E \multimap \mathtt{h}, \mathtt{B}\ \ S)$. Again, there are two cases. Either (*i*) `h` is an asset or (*ii*) it is a formal parameter. The code of cases (*i*) and (*ii*) is:

```
int tmp = E ;
//@ assert h-tmp >= 0 ;
h = h-tmp ;
B_h = B_h+tmp ;
Stipula2Java_A(S)
```

Case *(i)*

```
int tmp = E ;
//@ assert h-tmp >= 0 && A-tmp >= 0 ;
h = h-tmp ; A = A-tmp ;
B = B+tmp ;
Stipula2Java_A(S)
```

Case *(ii)*

In case (*ii*), we decrease and increase the assets `A` and `B`, respectively, where `tmp` is a fresh variable. In case (*i*), we assume that `h` is an asset, not a formal parameter. This allows us to determine what asset of `B` must be increased by the move (the general case is dealt with *ad hoc* annotations).

– $\textsc{Stipula2Java}_{\mathtt{A}}(\mathbf{if}\,(E)\,\{\,S'\,\}\,\mathbf{else}\,\{\,S''\,\}\ S) \stackrel{\text{def}}{=}$

```
if ( E ) { Stipula2Java_A(S') }
else { Stipula2Java_A(S'') }
Stipula2Java_A(S)
```

The semantics of the conditional is obvious: we simply use the corresponding `Java` statement.

Regarding the shortcuts $\mathtt{h} \multimap \mathtt{h}'$ and $\mathtt{h} \multimap \mathtt{A}$, we generate optimized `Java` code, to avoid the declaration of the auxiliary variable `tmp`:

$\textsc{Stipula2Java}_{\mathtt{A}}(\mathtt{h} \multimap \mathtt{h}') \stackrel{\text{def}}{=}$ `h'= h'+h ; h = 0 ; // h and h' assets`

$\textsc{Stipula2Java}_{\mathtt{A}}(\mathtt{h} \multimap \mathtt{h}') \stackrel{\text{def}}{=}$

```
//@ assert A_h'-h >= 0 ; // h formal param., h' asset
h'= h'+h ;
A_h'= A_h'-h ;
h = 0 ;
```

and $\textsc{Stipula2Java}_{\mathtt{A}}(\mathtt{h} \multimap \mathtt{B}) \stackrel{\text{def}}{=}$ `B_h = B_h+h ; h = 0 ; // h asset`

$\textsc{Stipula2Java}_{\mathtt{A}}(\mathtt{h} \multimap \mathtt{B}) \stackrel{\text{def}}{=}$

```
//@ assert A-h >= 0 ; // h formal parameter
B = b+h ; h = 0 ;
```

References

1. The Accord Project: Open source software tools for smart legal contracts, https://accordproject.org, accessed on March 31, 2026
2. Ahrendt, W., Beckert, B., Bubel, R., Hähnle, R., Schmitt, P.H., Ulbrich, M. (eds.): Deductive Software Verification: The KeY Book, LNCS, vol. 10001. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-49812-6
3. Ahrendt, W., Bubel, R.: Functional verification of smart contracts via strong data integrity. In: ISoLA (3). LNCS, vol. 12478, pp. 9–24. Springer (2020). https://doi.org/10.1007/978-3-030-61467-6_2
4. Beckert, B., Bubel, R., Drodt, D., Hähnle, R., Lanzinger, F., Pfeifer, W., Ulbrich, M., Weigl, A.: The Java verification tool KeY: A tutorial. In: Platzer, A., Rozier, K.Y., Pradella, M., Rossi, M. (eds.) Proc. 26th Intl. Symp. on Formal Methods, Milan, Italy. LNCS, vol. 14934, pp. 597–623. Springer, Cham (Sep 2024). https://doi.org/10.1007/978-3-031-71177-0_32
5. Beckert, B., Herda, M., Kirsten, M., Schiffl, J.: Formal specification and verification of hyperledger fabric chaincode. In: 3rd Symp. on Distributed Ledger Technology (SDLT), Gold Coast, Australia, November 12, 2018. pp. 44–48. Institute for Integrated and Intelligent Systems (2018)
6. Cassez, F., Fuller, J., Quiles, H.M.A.: Deductive verification of smart contracts with Dafny. Int. J. Softw. Tools Technol. Transf. **26**(2), 131–145 (2024). https://doi.org/10.1007/S10009-024-00738-1
7. Cok, D.R.: JML and OpenJML for Java 16. In: Cok, D.R. (ed.) FTfJP 2021: Proc. 23rd ACM Intl. Workshop on Formal Techniques for Java-like Programs, Virtual Event, Denmark. pp. 65–67. ACM (2021). https://doi.org/10.1145/3464971.3468417
8. Crafa, S., Laneve, C., Sartor, G., Veschetti, A.: Pacta sunt servanda: Legal contracts in *Stipula*. Science of Computer Programming **225**, 102911 (2023). https://doi.org/10.1016/j.scico.2022.102911
9. Filliâtre, J., Marché, C.: The Why/Krakatoa/Caduceus platform for deductive program verification. In: Damm, W., Hermanns, H. (eds.) Computer Aided Verification, 19th Intl. Conf., CAV, Berlin, Germany. pp. 173–177. LNCS, Springer (2007). https://doi.org/10.1007/978-3-540-73368-3_21
10. Freitas, A.F., Cavalcanti, A.: Automatic translation from *Circus* to Java. In: Misra, J., Nipkow, T., Sekerinski, E. (eds.) Formal Methods, 14th Intl. Symp. on Formal Methods, Hamilton, Canada. LNCS, vol. 4085, pp. 115–130. Springer (2006). https://doi.org/10.1007/11813040_9
11. Hähnle, R., Laneve, C., Veschetti, A.: Formal verification of legal contracts: A translation-based approach. In: Damiani, F., Farrell, M. (eds.) Integrated Formal Methods, 20th Intl. Conf., iFM, Paris, France. LNCS, vol. 16194, pp. 59–78. Springer, Cham (2025). https://doi.org/10.1007/978-3-032-10794-7
12. Hamie, A.: Translating the Object Constraint Language into the Java modelling language. In: Haddad, H., Omicini, A., Wainwright, R.L., Liebrock, L.M. (eds.) Proc. of the ACM Symp. on Applied Computing (SAC), Nicosia, Cyprus. pp. 1531–1535. ACM (2004). https://doi.org/10.1145/967900.968206
13. Laneve, C.: The Stipula Platform: A Workbench for Programming and Analyzing Legal Contracts. In: Journeys Between Formal Methods and the Railway Industry: Essays Dedicated to Alessandro Fantechi on the Occasion of His 70th Birthday. LNCS, vol. 16470, pp. 160–177. Springer (2026)

14. Lexon Foundation, https://gitlab.com/lexon-foundation, accessed on March 31, 2026
15. Merigoux, D., Chataing, N., Protzenko, J.: Catala: A programming language for the law. Proc. ACM Program. Lang. **5**(ICFP), 1–29 (Aug 2021). https://doi.org/10.1145/3473582
16. Nehaï, Z., Bobot, F.: Deductive proof of industrial smart contracts using Why3. In: Sekerinski, E., Moreira, N., Oliveira, J.N., Ratiu, D., Guidotti, R., Farrell, M., Luckcuck, M., Marmsoler, D., Campos, J.C., Astarte, T., Gonnord, L., Cerone, A., Couto, L., Dongol, B., Kutrib, M., Monteiro, P., Delmas, D. (eds.) Formal Methods. FM Intl. Workshops, Porto, Portugal, Revised Selected Papers, Part I. pp. 299–311. LNCS, Springer (2019). https://doi.org/10.1007/978-3-030-54994-7_22
17. Obsidian: A safer blockchain programming language, http://obsidian-lang.com/, accessed on March 31, 2026
18. OpenLaw: Real world contracts for Ethereum, https://www.openlaw.io, accessed on March 31, 2026
19. Roche, N., Hernandez, W., Chen, E., Siméon, J., Selman, D.: Ergo – a programming language for smart legal contracts. arXiv CoRR (Dec 2021). https://doi.org/10.48550/arXiv.2112.07064
20. Solidity Documentation: State Machine Common Pattern, https://docs.soliditylang.org/en/v0.8.35-pre.1/common-patterns.html#state-machine, accessed on March 31, 2026
21. The Stipula Project, https://github.com/stipula-language, accessed on March 31, 2026